

Lab 1:- Python Programming Fundamentals

Q.1) Write program to illustrate variables, constants, datatype and type conversion.

Theory:- Variables in Python

- Variables are used to store data values. You can say that it is a ~~label~~ ^{label} that you can assign a value to it. A variable is always associated with a value.

It can be re assigned to different values and types during programs execution, providing flexibility in how data is handled.

- Some key points about variables:

- Dynamic typing: In Python we don't need to declare variable before using them or declare their type. The type is assigned based on the value you assigned to the variable.

- No need to declare variable type: Unlike other languages like Java, C, C++, there is no need to declare the type of variable before using it.

- Naming Convention: Python variable must start with letter or an underscore, and can contain letter, numbers, and underscore.

In Python you can assign different types of values to the same variable at different points in the program.

Syntax:

variable-name = value; variable = expression

Example: Program 1:- Demonstration of variable in Python
[variable.py] File name

```
name = "Sarjan Rai"
```

```
age = 24
```

```
Address = "Sundarharaiicha - 10, Morang"
```

```
print("Hello!, Mr. {name}. Please correct it I am wrong  
you are {age}y. And your permanent address is  
{address}y.")
```

Output: Hello! Mr. Sarjan Rai. Please correct it I am wrong.
You are 24. And your permanent address is Sundarharaiicha - 10, Morang.

theory 2: Constants

In programming the term "constant" refers to named representing values that don't change during program's execution. Constants are a fundamental concept in Python, whose values are meant to remain unchanged ~~through~~ throughout the execution of a program.

It doesn't have a dedicated syntax for constants, but constants are written in uppercase. Constants are used for values that do not change, such as mathematical constants or configuration settings.

Syntax:

constant_name[uppercase] = value #unchanged.

Example:

```
API_TOKEN = "194946230967"
```

```
PI = 3.142
```

key points about constants

- Immutability by Convention: Constants are not truly immutable in Python, but they are treated as such by convention.
- Uppercase Naming: Constants are usually written in uppercase with underscores separating words like `DEFAULT_BG`.

Using constants you can make your code easier to read and maintain, as it's clear which values should not change. And you can also define global constants that can be used across different parts of same program.

Program 2: Demonstration of Constants

```
E = 2.718
```

```
DATABASE_URL = "localhost:8000/studentDB"
```

```
API_ENDPOINT = "https://api.pyconstant.com/example/"
```

```
ERROR_CODE_NOT_FOUND = 404
```

```
IS_DEBUG_MODE = True
```

```
ERROR_MESSAGE = "An unexpected error has occurred."
```

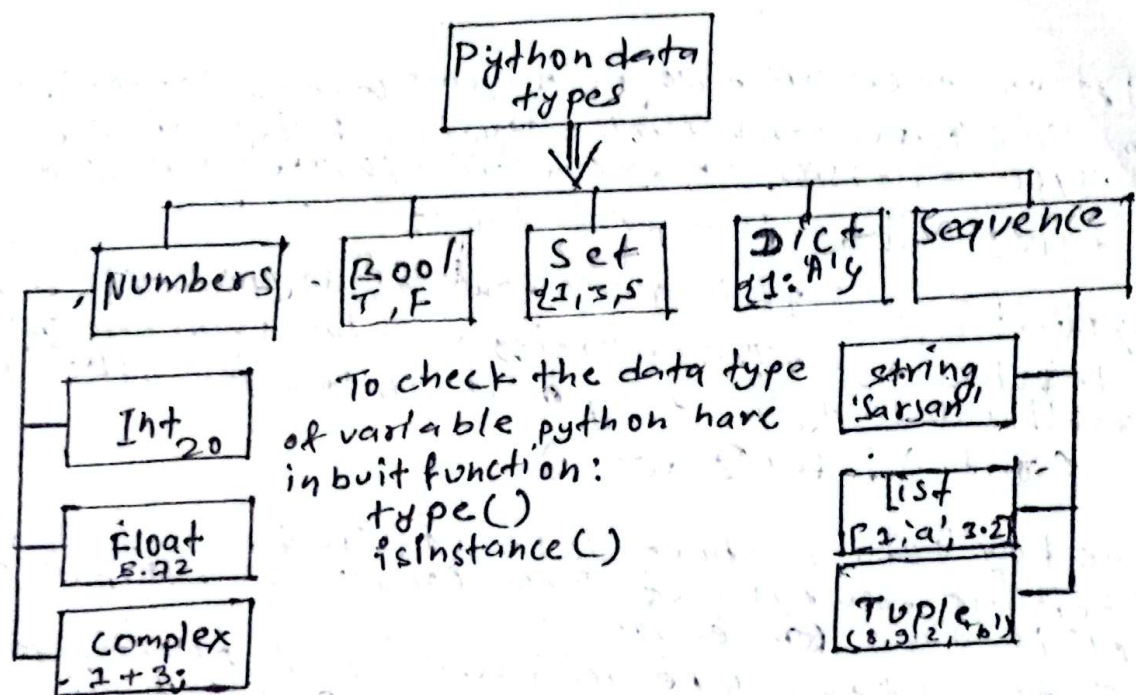
```
CONFIG_FILE_PATH = "/core/myapp/config.yaml"
```

```
SERVER_IP = "192.168.1.48"
```


Theory 3: Data types in Python

- Introduction: In python, a datatype is a classification that specifies the type of value a variable can store. In every programming world, data type is an important concept.

In simple terms data type specifies the different size and values that can be stored in the variable. As I mention data type in program is important because it determines what operations can be performed on the data and how data is stored in memory. Python is a dynamically-typed language, meaning you don't need to declare the type of variable explicitly - python determines it automatically based on the value assigned.



- Key points about Data types in Python
 - * Dynamic Typing: Variables type in python are determined at runtime and can change as the variable's value change.
 - * Type Flexibility: Python allows the type of a variable to change by simply assigning a different value.
 - * Built-in Types: Python have a variety of built-in datatypes like Integers, floats, strings, lists, dictionaries, tuples, etc.
 - * Immutability: Python have some datatypes like Strings and tuples are immutable, meaning their values can't be changed after they are created.

Data types in python are essential because they define the nature of the data a variable can hold and determine how that data can be manipulated. The use cases of data types in Python are broad and cover various programming scenarios, from simple arithmetic operations to complex data processing.

Theory 4: Type Conversion

Introduction: Every value in Python has a data type. As we discuss ~~pre~~ previously about data type in python, some time you need to change the type of the data to make it easier to use and work with it.

Type conversion in python refers to the process of change data type of a value or variable. In python it is also known as type casting or type coercion. Which are an important aspect of programming. Type conversion allow us to convert data from one type to another, this is often necessary when you need to perform operations that require specific data type or you want to convert between different representations of data.

In Python, there are two main types of the conversions:

- * Implicit Type conversion: Python automatically handles type conversion in expression involving mixed data types like integer to float

- * Explicit Type: It is also known as type casting, that involves manually converting a value from one type to another data type using built-in functions like `int()`, `float()`, `bool()`, `dec()`.

- Some key points about Type conversion

- * There are two types of Type conversion. Implicit and explicit type conversion.

- * Type safety can be achieved using explicit type conversion. It prevents runtime error and bugs.

- * Type conversion might lead to data loss or precision issues. Especially when different type of variable data type are being convert or that have different range.

Program: converting to integer

```
x_val = "29"
x = int(x_val)
print(type(x))
```

Output: <class 'int'>

int into string conversion

```
a_val = 460
y = str(a_val)
print('type:', type(y))
```

Output: <class 'str'>

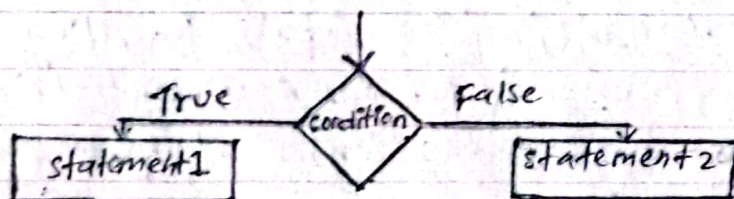
Lab 2

Q2.1: Control statements in Python

Q2.2: WAP to apply if, match, break & continue statements for decision making

Theory: Control statements direct the flow of the program's execution. It is designed to serve the purpose of modifying a loop's execution from its default behaviour.

Based on a condition, control statements are applied to alter how the loop's execution progresses. It allows to make dynamic decision during runtime with help of predetermined conditions. They introduce logic, decision-making, and looping and reusable functions.



Control Statements
in Python

Key control statements in Python include if/else, for/while, break/continue and pass. Together, they provide the mechanism to control the flow of a program and craft complex logic in a readable and maintainable fashion.

It is a fundamental component of programming that enable us to dictate the flow of execution within a program. And they are crucial for several reason:

Programs: demonstration of types of operations and performing calculation

Arithmetic operators

- $a = 10$
- $b = 5$
- $\text{print}(\text{"Addition:"}, a+b)$
- $\text{print}(\text{"Subtraction:"}, a-b)$
- $\text{print}(\text{"Multiplication:"}, a \times b)$

Output: Addition: 15
Subtraction: 5
Multiplication: 50

Comparison operators

- $x = 5$
- $y = 10$
- $\text{print}(\text{"Equal to operator:"}, x == y)$
- $\text{print}(\text{"Not Equal to Operator:"}, x != y)$
- $\text{print}(\text{"Greater than Operator:"}, x > y)$

Output:
Equal to operator: False
Not Equal to Operator: True
Greater than Operator: False

Logical Operators

- $i = \text{True}$
- $j = \text{False}$
- $\text{print}(\text{"Logical AND Operation:"}, i \text{ and } j)$
- $\text{print}(\text{"Logical OR Operation:"}, i \text{ or } j)$
- $\text{print}(\text{"Logical NOT Operation:"}, \text{not } j)$

Output:
Logical AND Operation: False
Logical OR Operation: True
Logical NOT Operation: True

Assignment Operators

- $p = 10$ # Assign value to a variable
- $\text{print}(\text{"value of p:"}, p)$

Output
value of p: 10

$p \times \times = 2$
 $\text{print}(\text{"Exponentiate and assign 'value of p:"}, p)$

Output
Exponentiate and assign 'value of p: 144

- $p + = 2$
- $\text{print}(\text{"Add and assign operator 'p:"}, p)$

Output
Add and assign operator 'p: 12

theory: Input and Output Functions in Python
In programming world, Input and Output operations are fundamental for interacting with users and processing data. In Python, I/O Functions allow you to read data from various sources and write data to different destinations.

In Python, program output is displayed using the `print()` function, while user input is obtained with the `input()`. Note that Python `input()` treats all input as strings by default, and requires explicit conversion for other data types.

* Input () Function
In simple terms, this function prints specified values and variables to the screen. To run a program or perform an operation with user input value, we use `input()` to get user input.

Syntax:

`input <key> (prompt)`

~~This function has another additional parameter like~~
This function has no other additional parameter like `print()`.

The prompt is optional. It is a string which is displayed in CLI that helps user for input operation.

- Characteristics of `input()`:

- * The "`input()`" function always returns user input as a string as default regardless of what the user type.
- * Unlike "`print()`", it has no other parameters for formatting, separating or other, only it takes an optional prompt parameter.

- * You can use the concept of type casting to convert data type as required.

- * Using `input()` you can take multiple inputs combined with methods like `split()`.

~~* `input()` Function~~

- * `print()` Function: `print()` is Python's built-in function. This is used to print or display output in screen, like console or in file.

theory: Input and Output Functions in Python
In programming world, Input and Output operations are fundamental for interacting with users and processing data. In Python, I/O Functions allow you to read data from various sources and write data to different destinations.

In Python, program output is displayed using the `print()` function, while user input is obtained with the `input()`. Note that Python `input()` treats all input as strings by default, and required explicit conversion for other data types.

* Input () Function
In simple terms, this function prints specified values and variables to the screen. To run a program or perform an operation with user input value, we use `input()` to get user input.

Syntax:

`input <key> (prompt)`

~~This function has another additional parameter like~~
This function has no other additional parameter like `print()`.

The prompt is optional. It is a string which is displayed in CLI that helps user for input operation.

- Characteristics of `input()`:
* The "`input()`" function always returns user input as a string, as default, regardless of what the user type.
* Unlike "`print()`", it has no other parameters for formatting, separating or other, only it takes an optional prompt parameter.

* You can use the concept of type casting to convert data type as required.

* Using `input()` you can take multiple input combined with methods like `split()`.

~~* Output () Function~~

* print () Function: `print()` is Python's built-in function. This is used to print or display output in screen, like console or in file.

The purpose of Output () Function : Output Functions are used to display data to the user or write data to file or console. And the usage of this function is to make output data to the console or another file-like object. The function can handle multiple arguments, like 'separators, end, and others

- object - value(s) to be printed
- sep - used to separate multiple objects inside print().
- end - used to add specific value at the end like new line "\n", tab "\t".
- file - where the values are printed the default value is sys.stdout
- flush - boolean, specifying if the output is flushed or buffered. And the default is false

The important uses of those input and print functions are

- user interaction: input(), print() makes interactive application, where user can provide data and receive output.
- data processing: They enable the collection of data which can be processed or stored, and the result can be displayed.
- debugging: Output functions 'print()' are often used for debugging; allowing developers to print variable value and program states.

programs : example of input, output function
name = input("Enter full Name: ")
print("Hello, " + name + "! How are you?")

Run code :-

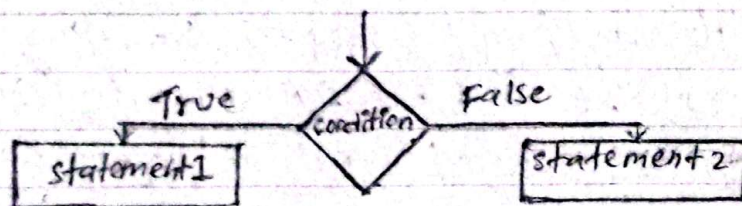
Enter Full Name: Sargan Rai

Output: Hello, Sargan Rai! How are you?

Q2.2 WAP to apply if, match, break & continue statements for decision making

Theory: Control statements direct the flow of the program's execution. It is designed to serve the purpose of modifying a loop's execution from its default behaviour.

Based on a Condition, control statements are applied to alter how the loop's execution progresses. It allows to make dynamic decision during runtime with help of predetermined conditions. They introduce logic, decision-making, and looping and reusable functions.



Control Statements
in Python

Key control statements in Python include if/else, for/while, break/continue and pass. Together, they provide the mechanism to control the flow of a program and craft complex logic in a readable and maintainable fashion.

It is a fundamental component of programming that enable us to dictate the flow of execution within a program. And they are crucial for several reason :

importance of Control statements in python

a) ~~Decide~~

a) decision Making: Allow ^{programs} to make decision based on user input or other variables, enabling more interactive application

b) Code Efficient: By using it, we can avoid code redundant, leading to cleaner and more efficient program

c) Flexibility: Control statements provide the ability to adopt the behaviour of a program in response to changing conditions or inputs, enhancing its versatility.

d) Error Handling: Exception handling is vital for managing unexpected errors and controlling program flow during these situations, helping to maintain robustness in applications.

Types of Control statements

A) *if statement: The simplest and most used form of conditional statement is the 'if' statement.

The if statement checks whether a given condition is true. If it evaluates to 'True', the associated block of code will execute.

Code:

```
if dresscode == "casual":  
    print("You are required to wear suit and tie")
```


B) Match

if else condition: The if-else, statement checks the condition and executes the 'if' block of code, when condition is 'True' and execute 'else' block of code if the condition states 'false'.

Syntax:

if condition:

statement 1

else:

statement 2

example code:

```
user_password = input("Enter password")
```

```
c_password = "XT18@#ac"
```

```
if user_password == c_password:
```

```
    print("Correct password")
```

```
else:
```

```
    print("please enter correct password")
```

Output:

Enter password: this is password

please enter correct password

output 2:

Enter password: XT18@#ac

Correct password

Match : The match statement in python is a control flow statement that allows you to match a value against multiple patterns and execute specific code block based on which pattern matches the value.

- In simple terms the match statement allows us to execute a block of code among many alternatives. The match / case is used to implement switch-case like characteristics ~~and~~ ~~it else functionalities~~ functionalities. It was firstly introduced in python version 3.10. The 'match' key word starts a block of code that takes an ~~an~~ expression as arguments (any valid python object, like variable, literal, a function call etc). 'case' key word introduces a pattern that is compared to the passed expression. The pattern can be object, ~~such as variable~~ or combination of objects using operators like or, as (alias), and - (wildcard).
- If the pattern matches the expression, the statements following the ':' are executed. The statements can be any valid print(), return, break, etc.
- 'other / _' key word is used to specify a default case that is executed if none of the other patterns match.

Syntax of Match Statement:

Match variable name:

case 'pattern 1': statement 1

⋮

case 'pattern n': statement n:

Code example:

```
def handle_response(status_code):
```

```
    Match status_code:
```

```
        case 200:
```

```
            return "OK"
```

```
        case 404:
```

```
            return "Page Not Found"
```

```
        case 500:
```

```
            return "Internal Server Error"
```

```
        case other:
```

```
            return "Status-detail not found"
```

```
print(handle_response(500))
```

Output: Internal Server Error

- c) continue statement: Another control statement. continue statement is used to end the current iteration in a for loop (or a while loop) and continue to the next iteration.

In short
Continue skips the current iteration and proceeds to the next one.

Working of continue statement in python

```
→ for val in seq:  
    if condition:  
        continue
```

Continue statement is just opposite to break statement, syntax of continue statement:

Looping statement:
Condition check:
Continue

Code Example

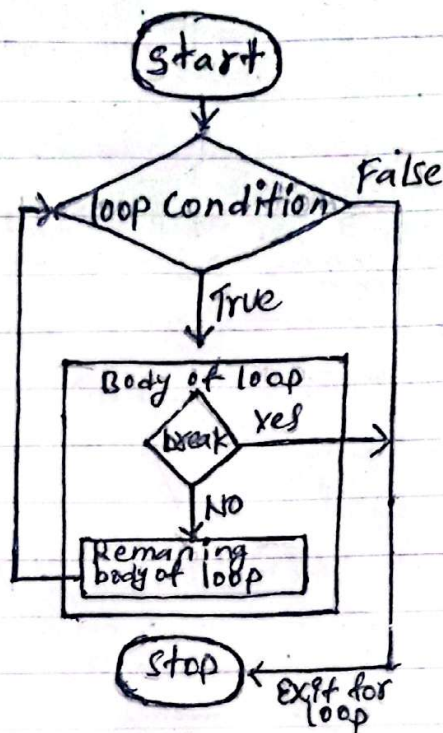
```
for i in range(1, 10, 2):  
    if i == 5:  
        continue # it will skip the rest of the loop  
    print(i)      # for i = 5 condition.
```

output:

1
3
7
9

①) Break : Break in python is a control flow statement and it play a crucial role in modifying the flow of execution within loops and conditional statements .

The Break statement is used to exit a loop prematurely ; regardless of whether the loop condition is still true.



Flow chart of a break statement

code example :

```
inventory_list = ["apple", "banana", "cherry", "grape",  
                  "orange"]  
search_item = "cherry"  
for item in inventory_list:  
    if item == search_item:  
        print("search item is found")  
        break
```


else:
print(f "a search item not found!")

out put:

'Cherry' found

Difference between control statements

Feature	if, else, elif	Match Statement	break Statement	Continue Statement
Purpose	used for decision making based on condition	used for pattern matching on values or patterns	exits the loop prematurely	skips the current iteration of a loop
Use Case	General decision making Multi branch logic	Simplifies handling multiple conditions based on pattern matching	To terminate the current loop iterations	To skip the current loop iteration
Syntax	if condition: elif elif condition: else:	Match value: case pattern: case - :	break	Continue
Control Flow	conditional execution	value or pattern matching	exits from loop's like for & while	skips to the next iteration in loops
Limitations	can become complex with many condition	Limited to Python version 3.10 and above ver.	May cause loops to exit prematurely	can make loops difficult to follow if overused.
Example	if x == 12: print("12") elif x == 22: print("Hey") else: print("Goodbye")	def class_age: Match Age: case Age if Age < 13: return "child" case Age if Age >= 60: return "senior" case -: return "invalid" print(f"you are {Age}")	for i in range(10): if i == 5: break print(i)	for i in range(20): if i == 5: continue print(i)

Syntax of Match statement in python

Match variable name:

case 'pattern 1': statement 1

:

case 'pattern n': statement n

Code example:

```
def handle_response(status_code):
```

```
    Match status_code:
```

```
        case 200:
```

```
            return "ok"
```

```
        case 404:
```

```
            return "Page Not Found"
```

```
        case 500:
```

```
            return "Internal server error"
```

```
        case other:
```

```
            return "status-detail not found"
```

```
print(handle_response(500))
```

Output: Internal server error

- 1) Continue statement: Another control statement. Continue statement is used to end the current iteration in a for loop (or a while loop) and continue to the next iteration.

In short
Continue skips the current iteration and proceeds to the next one.

Working of continue statement in python

```
→ for val in seq:  
    if condition:  
        continue
```

Continue statement is just opposite to break statement, syntax of continue statement:

Looping statement:
Condition check:
continue

Code Example

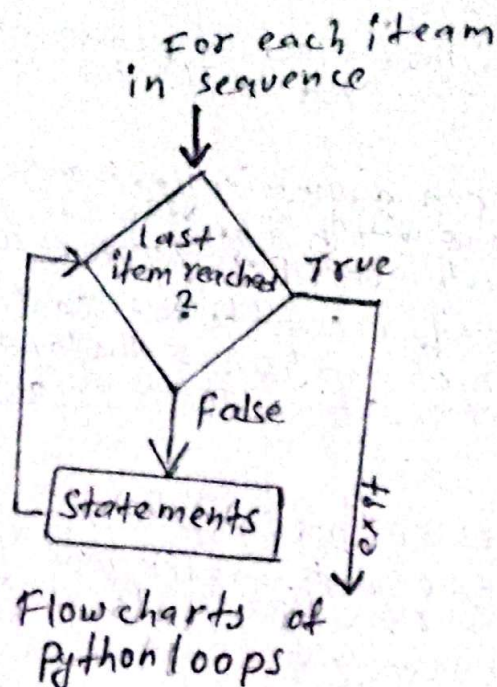
```
for i in range(1, 10, 2):  
    if i == 5:  
        continue # It will skip the rest of the loop  
    print(i)      # for i = 5 condition.
```

output:

1
3
7
9

theory: Loops statements in python

- Loops are fundamental control structures in programming that allow us to repeat a block of code multiple time based on some conditions and continue the tasks until it reaches certain conditions.



A loop is a programming construct that repeats a block of code as long as a certain condition is met. Loops help in performing repetitive task efficiently. python provides three types of looping statements or techniques:

- a) For loop
- b) while loop
- c) Nested Loops.

a) **for loop**: For loops are used for sequential traversal. This means they are specifically designed to iterate over elements in a sequence, such as a list, string or any iterable object in the order they appear. It automatically handles the iteration allowing each elements to be accessed systematically, making 'for' loops ideal for tasks like processing or searching through collections. Syntax:

for iterator_var in sequence:

b) **while Loops**: Using loops inside another loop is called nested loop. This code uses nested for loops to iterate over two ranges of numbers and prints the value of 'i' and 'j' for each combination of the two loops. If the range of 'i' is 2 to 3 and 'j' range is 1 to 3, then the inner 'j' loop continue for 3 times for each 'i' iteration.
 4,1; 4,2; 2,1; 2,2

Syntax:

for iterator_var in sequence:

for iterator_var in sequence:
statements(s)

statements(s)

- important of 'while' loop
while loops allow block of code to execute as long as a given condition is 'True' make this loop for indefinite iteration, loop.

* This loop can run dynamically based on dynamic condition, that can change within the loop itself. It can do complex logic with each loop's particular result.

* This loop can easily use different controlled statements like 'break' and 'continue' which gives the loop finer control over the flow of the loop makes this loop flexible.

c) Nested loops: A nested loop in Python is a loop inside another loop. If there are two loops, outer loop 'i' and inner loop 'j', then the outer loop 'i' runs first and each iteration of the 'i' the inner loop 'j' runs completely.

It runs for multi-dimensional data structure, such as matrices, or when solving problems that required multiple levels of iteration, like generating combinations, patterns or handling complex data processing tasks. Syntax:

for iterating_var in sequence:

for iterating_var in sequence:

execute your code

execute code

program:

```
1) for i in range(1, 10):  
    print(i)
```

output:

1
2
3
4
5
6
7
8
9

```
num = input 0;  
2) while True:  
    num = int(input("Enter Even Number: "))  
    if num % 2 == 0:  
        print("That is correct Even Number")  
        num  
        break
```

else:

```
print("Please, Enter correct Even Number:")  
exe: Enter Even Number: 4  
output: That is correct Even Number 4
```

```
3) Nested For loop  
rows = 5
```

```
for i in range(2, rows + 1):  
    for j in range(1, i + 1):  
        print("*", end=" ")  
    print()
```

output:

```
*  
* *  
* * *  
* * * *  
* * * * *
```


Q. Write a program to demonstrate input validation using loop.

while True:

```
user_input = int(input("Enter Even Number: "))  
if user_input.isdigit() and user_input % 2 == 0:  
    print("Thank you! You Entered: ", user_input)  
    break
```

else:

```
print("Invalid input. Please enter a even integer.")
```

exe: Enter Even Number: 3

Invalid input. Please enter a even integer: 2.

Thank you! You Entered 2.

Q. Write a program to create different patterns using nested loop:

pattern 1:

rows = 5

```
for i in range(1, rows + 1):
```

```
    for j in range(1, i + 1):
```

```
        print(i * j, end=" ")
```

```
    print()
```

output:

1

2 2

3 3 3

4 4 4 4

5 5 5 5 5

Pattern 2:

```
def print_ptt(size):  
    for i in range(size):  
        for j in range(size):  
            print("x", end=" ")  
        print()
```

print_ptt(5)

Output:

```
* * * * *  
* * * * *  
* * * * *  
* * * * *  
* * * * *
```

Pattern 3:

```
p_height = int(input("Enter Pyramid height: "))
```

```
for i in range(p_height):  
    for j in range(p_height - 1 - i):  
        print(" ", end=" ")  
    for k in range(2 * i + 1):  
        print("x", end=" ")  
    print()
```

exe:

Enter Pyramid height: 3

Output:

```
  *  
 * * *  
* * * * *
```


theory: Infinite loop in

Infinite loops are loops that run indefinitely until explicitly terminated. They are use full in scenarios where you want a program to keep running until a specific condition is met

program:

```
import psutil
import time

def m_system(threshold):
    while True:
        cpu_usage = psutil.cpu_percent(interval)
        memory_info = psutil.virtual_memory()
        print(f"CPU usage: {cpu_usage}%")
        print(f"Memory usage: {memory_info}%")

        if cpu_usage > threshold:
            print("Warning: High CPU usage detected")
        if memory_info.percent > threshold:
            print("Warning: High memory usage")

        time.sleep(5) // check every 5 sec
```

m_system(80)

output

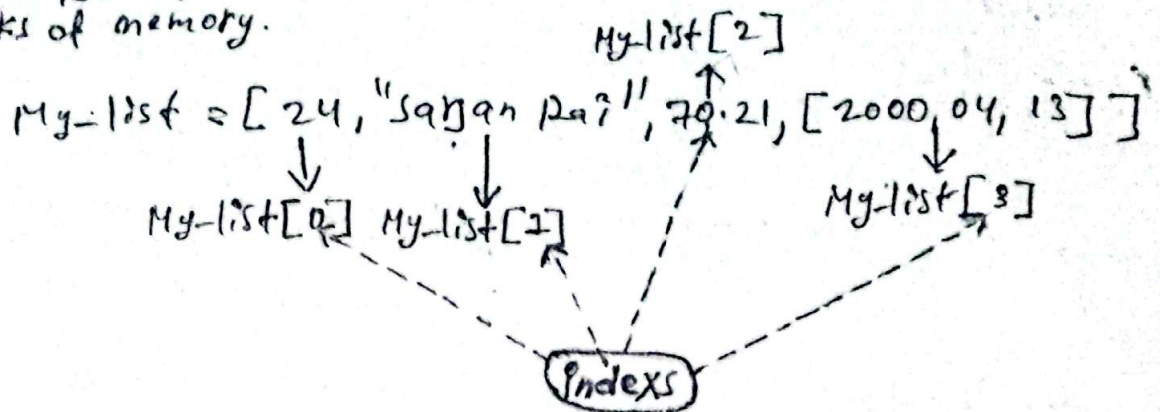
CPU usage: 5.8%
Memory usage: 32.1%

Lab 3: List, Tuple, dictionaries, set & strings.

Different between List, Tuple, set, dictionary and string

Feature	List	Tuple	Set	Dictionary	String
Def	An ordered mutable collection	An ordered collection of immutable elements	An unordered collection of unique elements	An unordered collection of key-value pairs	An ordered immutable sequence of characters
Syntax	My-list = [element1, element2, ...]	My-tuple = (element1, element2, ...)	My-set = {element1, element2, ...} or new-set = set([element1, element2, ...])	My-dict = {key: value, key: value, ...}	My-string = "Hi"
mutability	Mutable	Immutable	Mutable	Mutable	Immutable (cannot be change after creation)
Duplicates	Allowed	Allowed	Not Allowed	value can be duplicate not key	Strings are immutable (No duplication but characters can be repeated)
order preservation	ordered	ordered	unordered (insertion order preserved)	unordered but insertion order preserved	ordered (characters are stored as they are written)
Access method	indexing and sliceable	indexable and sliceable	only access via membership tests	Access using keys	indexable and sliceable (each character is an element)
Usage	use for dynamic modifiable collection	use for fixed collections of items	use for unique items and membership tests	use for mapping keys to corresponding values	use for handling text data.
key features	- versatile (supports multiple mutable data types)	- immutable good for constants data types	- Fast membership tests	- Enables fast lookup by keys	- Efficient for text manipulation and storage
Methods	append(), insert(), pop(), remove(), extend(), sort(),	count(), index()	add(), remove(), union(), clear(), pop()	get(), keys(), values(), update(), pop(), items(), clear()	upper(), lower(), replace(), split(), join(), find(), strip(), count()
performance	- slower than tuples & sets for membership tests - But faster for adding / removing elements	- Faster for iteration - memory efficient due to immutability	- Fast for iteration and membership tests - But slower for operations like sorting	- Extremely fast for lookups by key - But slower for unordered operations	- Efficient for text storage and manipulation - immutable nature aids optimizations

Q. WAP to create, add, remove and display list and list items.
 theory: The list class is fundamental built-in data type. It allows to efficiently organize and manipulate heterogeneous data. A list in Python contains values of different data types in contiguous blocks of memory.



properties of List

- **Ordered**: Maintain the order of the data insertion
- **Mutable/Changeable**: List is mutable and we can modify it
- **Heterogeneous items**: It can contain data of different types.
- **Duplication**: Allows duplicates data

Other characteristics of list are: Zero based, growable and dynamic, nestable, iterable, combinable, slicable.

List is one of the most versatile data types that can store multiple data types in single collection, it allow dynamic resizing and used for managing collections of related data.

* Advantages:

- easy to use and modify.
- Support heterogeneous data types.
- Build-in methods like sorting, reversing etc

* Disadvantages:

- It consumes more space than similar data types like arrays.
- Slightly slower for larger collection compared to tuples

Code example:

```
shopping-list = [
personal-info = ["Sargan Rai", 24, '65', True, ["Reading",
"Traveling"]]
print("Original Information:")
print(personal-info)
# Adding elements to profile
personal-info.append("BLC TE")
personal-info[4].append("Cooking")
print("Displaying Personal Information After adding
College Faculty And Hobby:")
```



```

print(personalInfo)
# Removing elements
personalInfo.remove(65)
print("\n displaying personalInfo after removing element: ")
print(personalInfo)

```

Q. WAP to make use of list slicing to display elements.
 theory: Slicing is a powerful feature in Python that allows to access specific parts of a list efficiently. It enables the selection of a subset of elements from a list without modifying the original list.

Syntax: `list[start: stop: step]`

- start: starting index (inclusive).
- stop: stopping index (exclusive).
- step: Interval between elements.
- '-1' Negative indexing allows slicing from end of the list. The very last element is refers by '-1' index.
- ':: -1' allows to achieve slicing with negative step, it will simply reverse the list.

* Advantage:

- Efficient to extract particular parts of a list.
- It has simple and intuitive syntax which makes it easy to use.
- It does not alter the original list.
- Support operations like: Reversing, skipping elements, or extracting specific patterns.

* Code Example:

```

my_list = ["Saran", 24, "student", 3.14]
print(f"displaying Name and Age: {my_list[:2]}")
print(f"Title is: {my_list[2:3]}")

```



```
print(f"Reversed : {MyList[::-1]}")
```

output:

displaying Name and Age: ["Sarjan", 24]

Title As: ["Student", 3.14]

Reversed: [3.14, "Student", 24, "Sarjan"]

9. WAPs to elaborate different list Method

append(): Adds a single element to the end of the list.

index(): Returns the index of the first occurrence of a value.

insert(): Add element at a specific position in the list.

sort(): Sorts the list in ascending order by default.

remove(): Removes the first occurrence of a value from the list.

reverse(): Reverses the order of element in the list.

min(): Returns the smallest element in the list.

max(): Returns the largest element in the list.

* code example:

```
user_profile = ["01", "Aakash Panday", "Software Developer",  
               "Python", 28, 192, 192428092, "married", "Nepal"]
```

```
user_profile.append(9812345678)
```

```
user_profile.insert(2, "aakashpanday@gmail.com")
```

```
index_of_name = user_profile.index("Aakash Panday")
```

```
print("Index of user Name: ", index_of_name)
```



```

numbers_list = [9, 2, 8, 100, 2119]
print("sorted_list:", numbers_list.sort())
min_num = min(numbers_list) # output min: 2
max_num = max(numbers_list) # output max: 100
user_profile.remove(192)
print(f"min: {min_num} & max: {max_num}")
print("Reversed user_profile", user_profile.reverse())

print("updated profile:", user_profile)

```

Output:

Index of user name: 1

updated profile: [01, "Aakash Panday", "aakash.panday@gmail.com", "softwaredeveloper", "python", 28, 192428.92, "married", "Nepal", 9812845678]

Max: 2

Min: 100

Q. WAP to apply List Comprehension

Theory: List Comprehension is a concise and efficient way to create lists in Python. It allows to generate lists in single line of code by applying an expression to each element of an iterable, optionally filtering the elements using a condition.

Syntax:

$\text{[Expression for item in iterable if Condition]}$
 (operation/value to be included in new list) (variable representing each element in iterable) (a collection of elements to loop through) (A filter to include only elements that meet certain criteria, it is optional)

→ Basic list comprehension

```
>>> [x for x in range(1,5)]
```

output

```
[1, 2, 3, 4]
```

⇒ List Comprehension with condition

```
>>> [x for x in range(1,10) if x%2 == 0]
```

output:

```
[2, 4, 6, 8]
```

⇒ Nested list comprehension

```
MNLC = [[j for j in range(3)] for i in range(4)]
```

```
print(MNLC)
```

output :

```
[[0, 1, 2], [0, 1, 2], [0, 1, 2], [0, 1, 2]]
```


§ wap to illustrate two-dimensional list and perform basic operation.

* Theory: Two-dimensional list, often used as matrix representations, are powerful data structures that allow programmers to organize data in a grid-like fashion. A 2D list in python is essentially a list of lists, where each inner list represents row of the matrix. It allows for easy representation and manipulation of tabular data.

Two-dimensional list is widely used in programming languages for task that involve matrices, tables, or spatial data representation.

• Structure: Rows and Columns in a grid-like format

Data type: Supported in many programming language like python, Java and C++.

* Key Features:

- Rows and Column: data is organized into rows and columns, providing a structured layout

- Indexing: Typically 0-based indexing is used to access elements
Element Access pattern: `My-list[row][column]`

- Dynamic Nature: The size of a 2D list can be changed dynamically by adding, removing rows & column.

- Nested Lists: A 2D list is implemented as list of lists, where each list represents a row.

- Versatility: It can store any type of data (strings, numbers or objects) within the grid.

* Advantage

- Efficient Data Organization: Efficiently store data in structured manner

- Easy Access: Element can be access directly using indexing (indices)

- Flexibility: Store different types of data and can be dynamically resized

- Visual Representation: Data can be represented visually; like "spatial problems" such as graphs, grids, or image.
- * Disadvantages:
 - Memory usage: Required more memory compared to lists for the same amount of data.
 - Complexity: Can be complex in operation like traversing or modifying because it required nested loops.
 - Fixed Rows & Column: In some language resizing may require additional memory management.
 - Error-prone: Indexing errors are common if row & column indices are not carefully managed.

* Usage

2D lists have wide range of application, including:

a) Matrices & Mathematical Operations:

- representing & performing operations on Matrices
- Operations: Transpose, Multiplication, addition of Matrices

b) Tables & Spreadsheet:

→ Storing data in tabular format for database or CSV processing.

c) Game Development:

- Game boards like chess, tic-tac-toe, or mine sweeper

d) Graphics & Images:

- Storing pixel values for image processing & spatial transformation

e) Geographical Data:

- Representing grids for maps or heatmaps.


```
two_d_list = [  
    [1, 2, 3],  
    [4, 5, 6],  
    [7, 8, 9],  
]
```

Accessing elements using index

```
ele = two_d_list[1][2] # output: 6  
print(ele)
```

Modifying elements

```
two_d_list[1][1] = 0 # modifying element 5 to 0  
print(two_d_list[1][1]) # output 0 previous element 5
```

Adding rows & column

```
two_d_list.append([10, 11, 12])
```

```
for r in two_d_list:
```

```
    r.append(0)
```

```
print(two_d_list) # output [[1, 2, 3, 0], [4, 0, 0, 0], [7, 8, 9, 0],  
                             [10, 11, 12, 0]]
```


9. WAP to create tuples, add elements, remove elements and display tuple items.

Theory:

Tuple is an ordered collection of immutable elements that can be of any data type, including strings, integers, numbers, lists or even other tuples. It uses small brackets '()' to define a tuple where each element is separated with a comma ','.

Syntax: name = (element1, element2, ...)

* Key features:

- Immutability: Once created, its elements can't be modified.
- Ordered: Elements in tuple are ordered and can be accessed using index.
- Allow duplication value: The element in tuple can be repeated.
- Heterogeneous data types: It can store mixed data types.
- Parenthesis: Tuples are enclosed in parentheses '()'.

* Advantages:

- Data Integrity: Maintain the integrity of data once tuples are created, it can't be modified, which makes it reliable data type where we need constant data.
- Faster Performance: It can be processed faster than list in some cases.
- Hashable: Tuple can be used as keys in dictionaries (unlike list).

* Disadvantage:

No Modification: It is less flexible compared to lists.
Lack of methods.

Creating a tuple

```
Org_tuple = ("sarjan", 24, 04, 6.2)  
print (new_tuple)
```

Adding Elements

```
new_tuple = (9, "seven-seh")  
Org_tuple += new_tuple
```

Removing Element

```
temp_tuple = list(Org_tuple)
```

```
temp_tuple.remove(3)
```

```
Org_tuple = temp_tuple  
tuple
```

#(Converting list back to tuple)

Displaying elements

```
# Org_tuple elements: "sarjan", 24, 04, 9, "seven-seh"
```

```
for element in Org_tuple:
```

```
    print (element, end=" ")
```


9 WAP to create, add, remove, display 'dictionary' items.
* theory:

Dictionaries are the data type / data structures that hold key-value pairs, which resemble real-world dictionaries. The unique keys are mapped to their corresponding values. It is one of the most fundamental data types, which are widely used to represent real-world data & structures.

Python dictionaries allow you to create association between two values. These associations are called "key-value pairs". Where each key is associated with its corresponding value.

Values in dictionary can be any data type and value can be duplicated, whereas keys can't be repeated and must be immutable. Python dictionary are ordered collection, which keys are case sensitive.

Dictionary internally use Hashing, hence, operations like search, insert, delete can be performed in Constant Time. Although dictionaries ~~are~~ are ordered but before version 3.6 and earlier python version it was unordered.

* Key features

- Mutable
- Key-value pair
- Ordered after python 3.7 version
- Dynamic type
- Allow duplication value, not keys
- Use curly brackets {}, each key-value pair are separated by a comma ',' and ^{each} key are paired with its value by using a colon ':'.
n

* Advantages

- Efficient data access: promote fast lookups using keys, and can be modified
- Dynamic Size
- Flexibility: Supports complex data models using nested dictionaries.

* Disadvantage:

- Memory Overhead: Required more memory compared to list or tuples.
- Key constraints: Key must be unique & immutable.

Basic operations of dictionary:

- creating a dictionary
- Adding elements.
- Removing elements
- display items.

creating

```
BICTE-STUDENTS = {  
    "Name": "Sarjan Rai",  
    "age": 24,  
    "symbol-no": 77214026 y
```

Adding

```
Additional = {  
    "sem": 7,  
    "Batch": 9 y
```

BICTE-STUDENT ~~=~~ BICTE-STUDENT | Additional

Removing element

```
remove = BICTE-STUDENTS.pop("age") # key 'age' and value 24  
# will be remove
```


displaying

```
B: for key, value in BICTE_STUDENT.items():  
    print(f"{key}: {value}")
```

output :

name: Sarjan Rai

Symbol_no: 77214026

sem : 7

Batch : 9

g WAP to make use of String Manipulation Method and perform different string operations.

```
string-demo = "Hello Nepal"
```

```
print(string-demo.lower()) # output: hello nepal
```

```
print(string-demo.upper()) # output: HELLO NEPAL
```

```
print(string-demo.title()) # output: HelloNepal
```

```
print(string-demo.capitalize()) # output: Hello.nepal
```

```
dem-str = "      space"
```

```
print(dem-str.strip()) # output: space.
```

```
RS-text = "Sargan,rai,koshi"
```

```
print(RS-text.replace("koshi", "sundar")) # output: Sargan,rai,sundar
```

```
print(RS-text.split(",")) # output: ["Sargan", "rai", "sundar"]
```

```
sep-text = ["Sargan", "Rai"]
```

```
print(" ".join(sep-text)) # output: Sargan Rai
```

```
Any-text = "Sargan24"
```

```
print(Any-text.isalnum()) # output: True
```

```
print(Any-text.isdigit()) # output: False.
```

```
print(Any-text.find("24")) # output: 6
```

other analyzing method

is space()

is upper()

is lower()

```
print(Any-text.count("a")) # output: 2 # search pass value in string and return occurrence number
```

```
print(len(Any-text)) # output: 8
```


Q WAP to create sets, add elements, remove elements and display set items. And perform set operations like union, intersection, difference.

* Theory:

A set refers to a collection of distinct objects. It is used to group objects together and to study their properties and relationships. The objects in a set are called elements or members of the set."

A set is an unordered collection of unique elements which is defined using curly braces `{ }` or the built-in `set()` function. Python sets are particularly useful for membership testing, removing duplicates from sequence, and performing common mathematical set operations like union, intersection, and difference. Set can be create with two different ways

By using curly braces $= \text{My-set} = \{1, 2, 3, 4, 5\}$
or By using `set()` function $= \text{newset} = ([3, 4, 3, 2, 2])$

* Key features of sets:

- Unordered
- Unique elements (no duplication)
- Mutable
- Fast membership testing using `in` or `not in`

* Common set operations:

- Union (`|` or `union()`): Combines elements from two sets
- Intersection (`&` or `intersection()`): Elements between two sets
- Difference (`-` or `difference()`): Elements present in one set but not in other.
- Symmetric Difference (`*` or `symmetric_difference()`): Elements unique to either set.

Creating a Set, Add Elements, Removing Elements & Display Items.

Org-set = {1, 2, 3}

Add elements

Org-set.add(4) # single element adding in set

Org-set.update([5, 6, 7, 8, 9])

Removing Elements

Org-set.remove(9) # different then discard; it raise error if element
not found. Passed as a parameter

Org-set.discard(8)

display items.

Org-set

for item in Org-set:
 print(item, end = " ")

Performing set Operations

A = {1, 2, 3, 4}

B = {4, 5, 6, 7, 8}

union

union-set = A | B

Alternative

or

union-set = A.union(B)

Output : 1, 2, 3, 4, 5, 6, 7, 8

intersection

inte-set = A & B

or inte-set = A.intersection(B)

output : 4

difference

dif-set = A - B

or dif-set = A.difference(B)

Output : 1, 2, 3, 5.